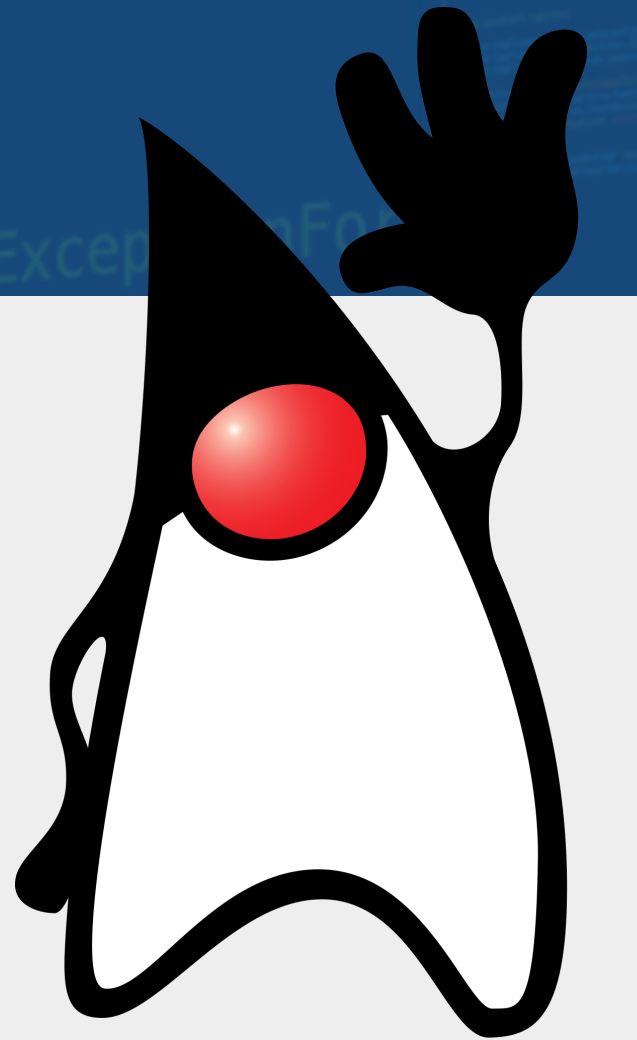


Java Programlama Dili

24. Bölüm enum

Akın Kaldıroğlu



Amaç



- Bu bölümde enum tipi incelenecektir



- **Değer Uzayı Problemi**
- **enum Tipi**
 - Kurucular
 - `java.lang.Enum`
 - Diğer Özellikler
- **Ödevler**
- **Bundan Sonra?**

Değer Uzayı Problemi



Problem - I



- Gün, ay ve yıldan oluşan bir tarihi ifade etmek üzere geliştirilmiş aşağıdaki sınıfları düşünün, sizce bu sınıfların en temel problemi nedir?

```
class Date1 {  
    int dayOfMonth;  
    int month;  
    int year;  
  
    public Date1(  
        int dayOfMonth,  
        int month,  
        int year) {  
        this.dayOfMonth = dayOfMonth;  
        this.month = month;  
        this.year = year;  
    }  
    ...  
}
```

```
class Date2 {  
    String dayOfMonth;  
    String month;  
    String year;  
  
    public Date2(  
        String dayOfMonth,  
        String month,  
        String year) {  
        this.dayOfMonth = dayOfMonth;  
        this.month = month;  
        this.year = year;  
    }  
    ...  
}
```

Date1 ve Date2



- `date`
- `Date1 ve Date2`

Problem - II



- Bu sınıfların durumunu oluşturan değişkenlerle ilgili birkaç problem vardır:
- Değişkenlerin tipleri problemlidir:
 - Ne `int` ne de `String`, değişkenlerin tabiatına uygun tipler değildir, çünkü bu değişkenlerin alabilecekleri değerler ile bu tiplerin alabilecekleri değer uzayı uyumlu değildir

Problem - III



- Alanların tipinin `int` olması durumunda negatif ya da sınırların dışında geçilebilecek değerler anlamsız olacaktır
- Ayrıca `int` kullanıldığında hangi değer hangi aya denk geldiği de açıklanmaya muhtaçtır
- `String` olması durumunda da “OCAK”, “Ocak” ve “ocak” gibi farklı yazımlar ile “Ock” gibi yanlış yazımlar söz konusu olacaktır

public static final Sabiteler - I



- Olması gereken bu gibi alanların alabileceği sınırlı sayıdaki tüm değerlerin **sabite** (**constant**) olarak tanımlı olmasıdır
- Java uzunca bir süre bu problemi, ilgili sınıflarda tanımladığı **public static final** sabitelerle yönetti

public static final Sabiteler - II



```
public class DateConstants {  
    // YEARS  
    public static final int YEAR_2026 = 2026;  
    public static final int YEAR_2027 = 2027;  
    public static final int YEAR_2028 = 2028;  
    ...  
    // MONTHS  
    public static final int JANUARY = 1;  
    public static final int FEBRUARY = 2;  
    ...  
    public static final int NOVEMBER = 11;  
    public static final int DECEMBER = 12;  
    // DAYS  
    public static final int DAY_1 = 1;  
    public static final int DAY_2 = 2;  
    ...  
    public static final int DAY_30 = 30;  
    public static final int DAY_31 = 31;  
}
```

```
class Date3 {  
    // Use DateConstants for int fields  
    int dayOfMonth;  
    int month;  
    int year;  
  
    public Date3(  
        int dayOfMonth,  
        int month,  
        int year) {  
        this.dayOfMonth = dayOfMonth;  
        this.month = month;  
        this.year = year;  
    }  
}
```

Date3



- `date.Date3`

public static final Sabiteler - III



- Örneğin
 - `java.sql.Connection`'da transaction isolation seviyeleri için 0, 1, 2, 4 ve 8 değerlerine sahip `int` sabiteler
 - `javax.servlet.http.HttpServletResponse`'taki HTTP cevap durumları olarak 100'lü, 200'lü, 300'lü, 400'lü ve 500'lü `int` sabiteler
- Geçerli değerlerin neler olduğu ilgili API'ye bakılarak öğrenilebilir

public static final Sabiteler - IV

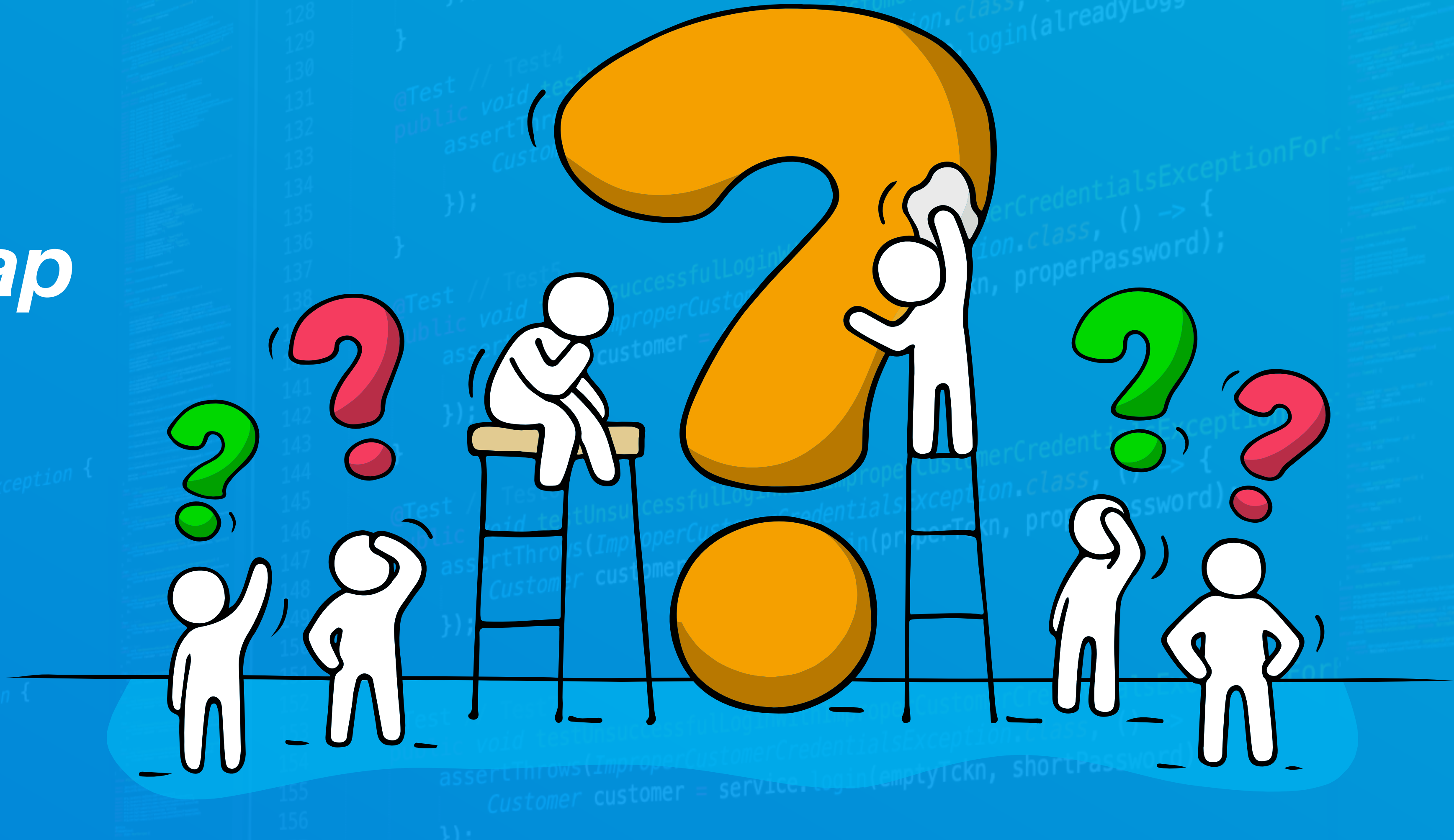


- Bir işin başlama tarihini ifade etmek üzere geliştirilmiş aşağıdaki sınıfları düşünün, sizce en temel problemi nedir?
- Hâlâ hataya açıklar!

```
public class DateConstants {  
    // YEARS  
    public static final int YEAR_2026 = 2026;  
    public static final int YEAR_2027 = 2027;  
    public static final int YEAR_2028 = 2028;  
    ...  
    // MONTHS  
    public static final int JANUARY = 1;  
    public static final int FEBRUARY = 2;  
    ...  
    public static final int NOVEMBER = 11;  
    public static final int DECEMBER = 12;  
    // DAYS  
    public static final int DAY_1 = 1;  
    public static final int DAY_2 = 2;  
    ...  
    public static final int DAY_30 = 30;  
    public static final int DAY_31 = 31;  
}
```

```
class Date3 {  
    // Use DateConstants for int fields  
    int dayOfMonth;  
    int month;  
    int year;  
  
    public Date3(  
        int dayOfMonth,  
        int month,  
        int year) {  
        this.dayOfMonth = dayOfMonth;  
        this.month = month;  
        this.year = year;  
    }  
}
```

Soru ve Cevap Zamanı!



enum Tipi



enum - I



- Değer uzayı problemini çözenin daha güzel bir yolu, **enumeration** (**sıralama**) yapısıdır
- Bunun için Java'ya 5 sürümüyle gelmiş **enum** anahtar kelimesi vardır
- **enum**, kendi tipinden, tanımında belirtilen belirli sayıda nesneyi sıralayan özel bir tiptir

```
enum Size {  
    SMALL, MEDIUM, LARGE, EXTRA_LARGE;  
}
```

enum - II



- **enum** tipi `public` ya da varsayılan erişime sahip olabilir
- **enum** nesneleri daima `public`, `static` ve `final`'dır yani sabitedir ve sadece kendi içinde tanımlanabilir
- Sabitelerin tipi, tanımlanan **enum**'ın tipidir ve tipin üzerinden erişilir
- Bu yüzden sabiteler büyük harfle yazılır ve gerekirse `_` içerir

```
public enum Day { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY;}
```

```
Day firstDay = Day.MONDAY;
```

enum - II



- **enum** tipinin sabiteleri , ile ayrılarak dizilir
- Sabiteler dışında başka bir şey yok ise tanımlamanın ; ile bitmesine gerek yoktur
- Varsa da ;'dan sonra başka sabite tanımlanamaz
- Sabiteler **enum** tipinin en başında, varsa diğer tüm alanlar ve metotlardan önce tanımlanmalıdır

```
enum Size {  
    SMALL, MEDIUM, LARGE, EXTRA_LARGE  
}
```

enum - IV



- Aynı sabite sadece bir defa tanımlanabilir
- **enum** sabiteleri sadece kendi içinde tanımlanır ve sabiteler için hiçbir niteleyici (modifier) kullanılamaz, zaten sabiteler, **public**, **static** ve **final**'dir
- Tamamen boş bir enum tanımlanabilir; sabitesi olmayan bir enum'ın alan ya da metodu varsa, gövdesi en azından bir `;` ile başlamalıdır



- **AnEnum**
- Sabitelerde hiçbir niteleyici kullanılamayacağını gözlemleyin
 - Bunun için sabitelerin başına **public**, **static** ve **final** vb. niteleyicileri yazın

EmptyEnum



- EmptyEnum

enum - V



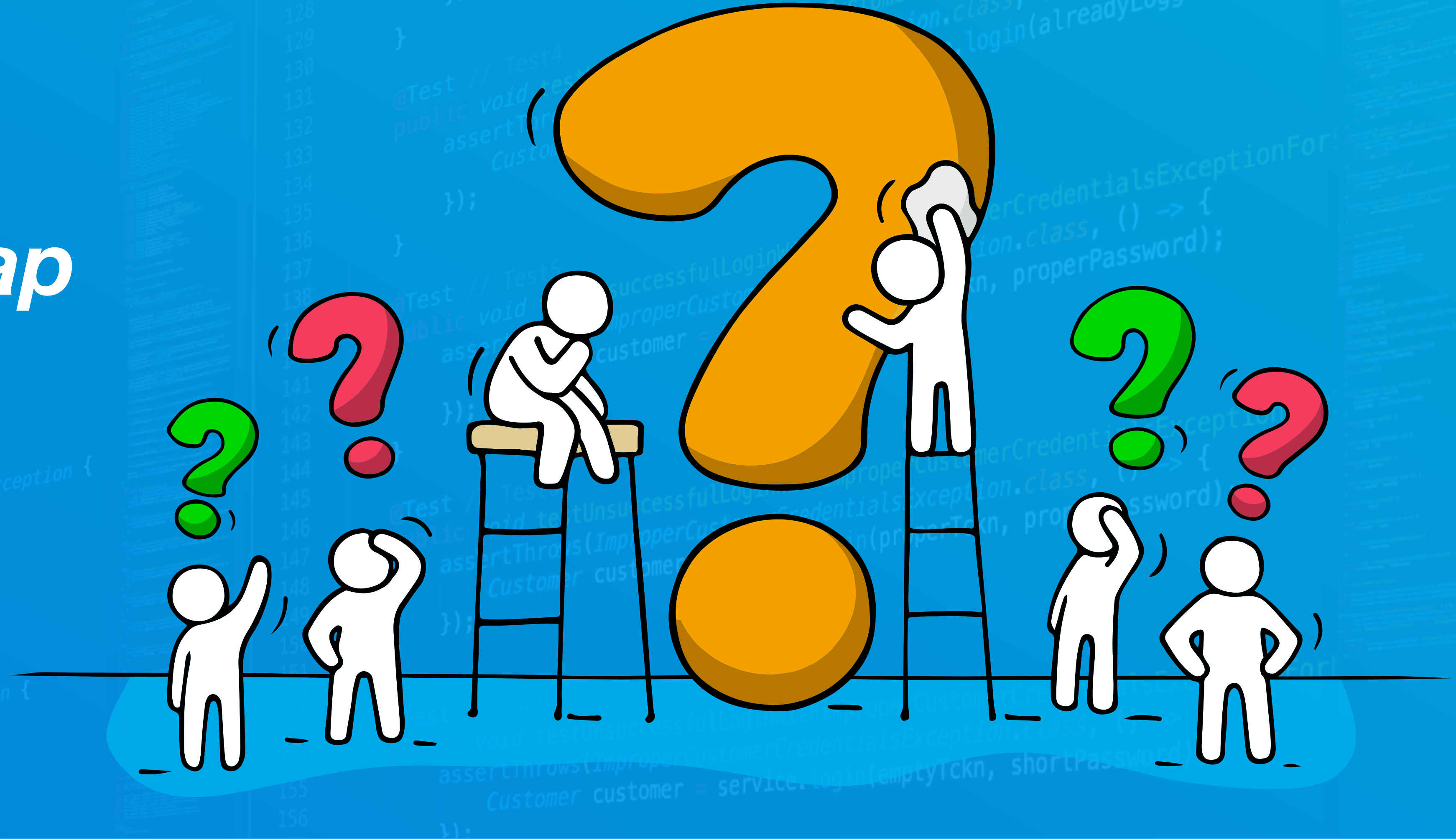
- `enum`, `main` metot tanımlayabilir ama `main` metodun `static` olması gereklidir, nesne `main` metodu hata verir

EnumWithMain



- `EnumWithMain`

Soru ve Cevap Zamanı!





Kurucular

enum Kurucuları - I



- **enum**, sınıflar gibi kuruculara sahip olabilir
- **enum** tipinin kurucuları sadece **private** erişim niteleyici alabilir; **public** ya da **protected** kullanımı bir hatadır
- Hiçbir niteleyici kullanılmadığı durumda yani varsayılan halde kurucular, paket erişimine değil, **private** erişime sahip olur
- Bir **enum** tipinin kurucusunu **private** yapmak gereksizdir

enum Kurucuları - II



- Dolayısıyla **enum** tipinin kurucuları, ne tipin içinden ne de dışından çağrılabilirler
- **enum** tipinin kurucuları sadece sabite tanımında kullanıldığı haliyle sınıf başlatılırken sistem tarafından çağrılır
- Bir de birden fazla kurucu varsa **this ()** ile çağrılabilirler

enum Kurucuları - III



- **enum**'da parametresiz sabite tanımında varsayılan kurucu çağrılır
- Sınıfta olduğu gibi **enum**'da da akıllı kurucunun olmadığı durumlarda varsayılan kurucu derleyici tarafından sağlanır
- Aşağıdaki iki **enum** da aynıdır

```
enum Size {  
    SMALL, MEDIUM, LARGE, EXTRA_LARGE;  
}
```



```
enum Size {  
    SMALL, MEDIUM, LARGE, EXTRA_LARGE;  
  
    Size(){}  
}
```

enum Kurucuları - IV



- **enum**'da sabiteler oluşturulurken uygun parametreler geçilecek şekilde akıllı kurucular tanımlanabilir
- **enum**'da sınıf ve nesne için özellik ve metotlar olabilir ve bunlara **enum** sabiteleri üzerinden erişilir

```
enum Size {  
    SMALL("S", 1), MEDIUM("M", 2),  
    LARGE("L", 3), EXTRA_LARGE("XL", 4), UNKNOWN;  
  
    private String abbreviation;  
    private int no;  
    private static String description = "These are sizes.";   
  
    Size() { // Called for UNKNOWN  
        this("?", 0);  
    }  
  
    Size(String abbreviation, int no) {  
        this.abbreviation = abbreviation;  
        this.no = no;  
    }  
  
    public String getAbbreviation(){ return abbreviation; }  
    public int getNo() { return no; }  
  
    public static String getDescription() {  
        return description;  
    }  
}
```

enum Tipi Başlatma



- **enum** tipi, diğer sınıflar gibi sınıf ve nesne başlatma bloklarına (initializer blocks) sahip olabilir
- **enum** tipi, bir üyesine ulaşıldığında başlatılır ve tüm sabiteleri oluşturulur
- **enum** tipinin en başında kendi tipinden statik sabiteler tanımlandığından, başlatmada önce nesne başlatma blokları, sonra kurucular, sonra da statik başlatma blokları çalışır
- Ayrıca nesne başlatma blokları ve kurucularda statik değişkenlere ulaşamaz

Size ve Sizes



- `size`
- `Size`
- Sadece `Sizes`'in `initialize()` metodunu çalıştırarak sonucu gözlemleyin

enum, Erişim ve Miras



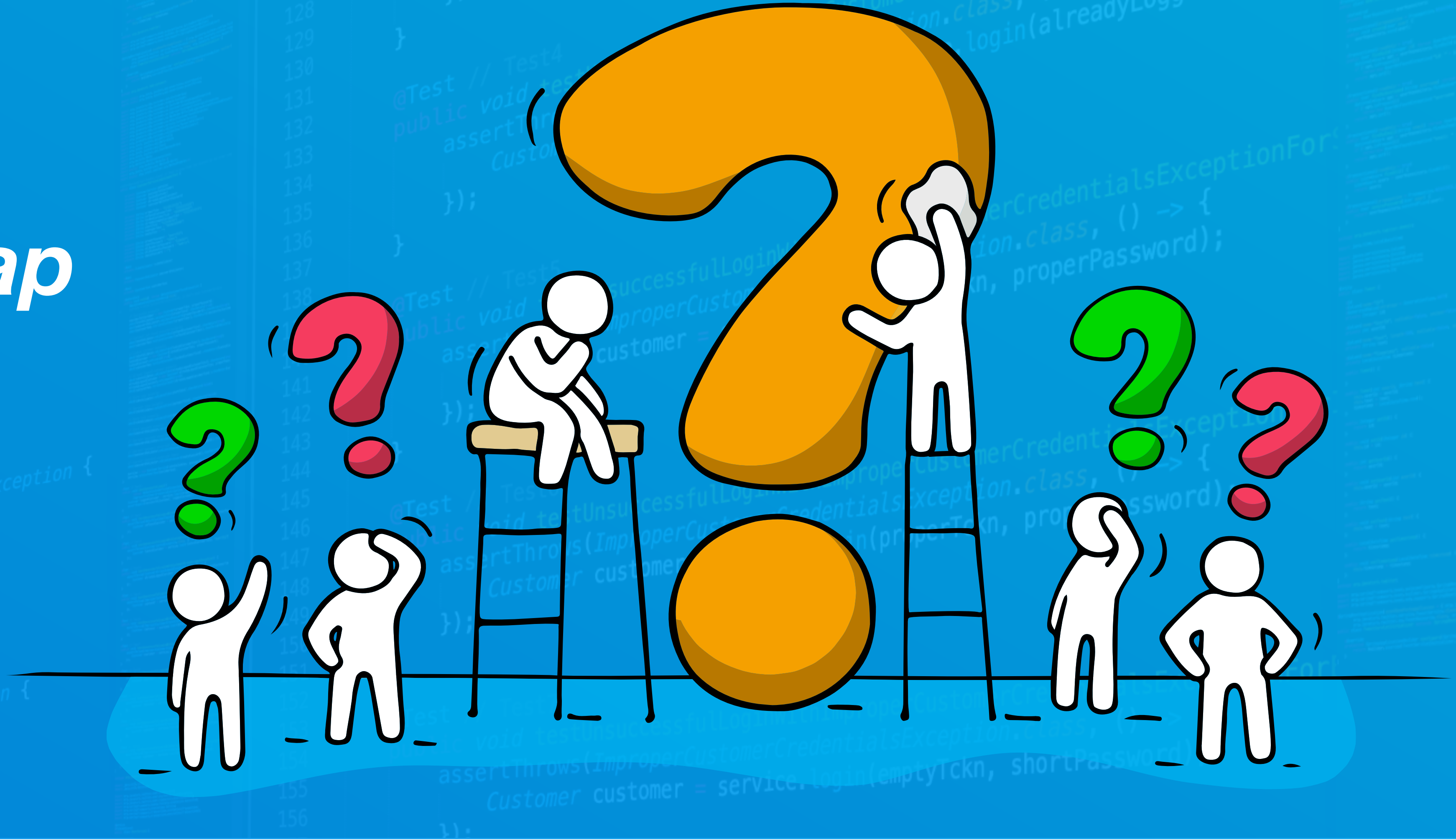
- **enum** tipi, sınıflar gibi `public` ya da varsayılan erişime sahip olabilir
- **enum** tipi, üstü kapalı bir şekilde (implicitly) `final`'dır dolayısıyla miras alınamaz, hiçbir başka tip tarafından üst tip olarak tanımlanamaz
- Ama **enum**'ı `final` tanımlamak bir derleme hatasıdır

AnotherEnum



- `AnotherEnum`

Soru ve Cevap Zamanı!





java.lang.Enum

java.lang.Enum Sınıfı



- Bütün **enum** tipleri, `java.lang.Enum` tipinin alt sınıfıdır
 - (Miras (inheritance) konusu bu eğitimin konusu değildir.)
- Bu sınıftan devralınan metotlardan ikisi:
 - `String name()`: **enum** nesnesinin `String` halini döndürür
 - `int ordinal()`: **enum** sabitesinin sırasıdır ve 0'dan başlar
- Bu metotlar tüm **enum** sabitelerinde çağrılabilir

values()



- Ayrıca **enum** tipler üzerinde çağrılacak bir de **static** metot vardır:
- `static T[] values()`: **enum** tipinde tanımlı tüm sabiteleri dizi olarak verir
- Buradaki **T**, **enum** ile tanımlanan ve `values()` metodunun üzerinde çağrıldığı tiptir

Size ve Sizes



- `size`
- `Size`
- `Sizes`'in `sizes()` metodu

Day ve Days



- `day`
- `Day`
- `Days`

Date Problemi



- Dolayısıyla baştaki **Date** problemi **enum** ile çözülebilir
- **Date** sınıfı için gerekli gün, ay ve yıllar birer **enum** olarak tanımlanabilir

```
public enum DayOfMonth {  
    DAY_1, DAY_2, DAY_3, DAY_4, DAY_5, DAY_6, DAY_7,  
    DAY_8, DAY_9, DAY_10  
}
```

```
public enum Year {  
    YEAR_2026, YEAR_2027, YEAR_2028, YEAR_2029, YEAR_2030  
}
```

```
public enum Month {  
    JANUARY, FEBRUARY, MARCH, APRIL, MAY, JUNE, JULY, AUGUST,  
    SEPTEMBER, OCTOBER, NOVEMBER, DECEMBER  
}
```



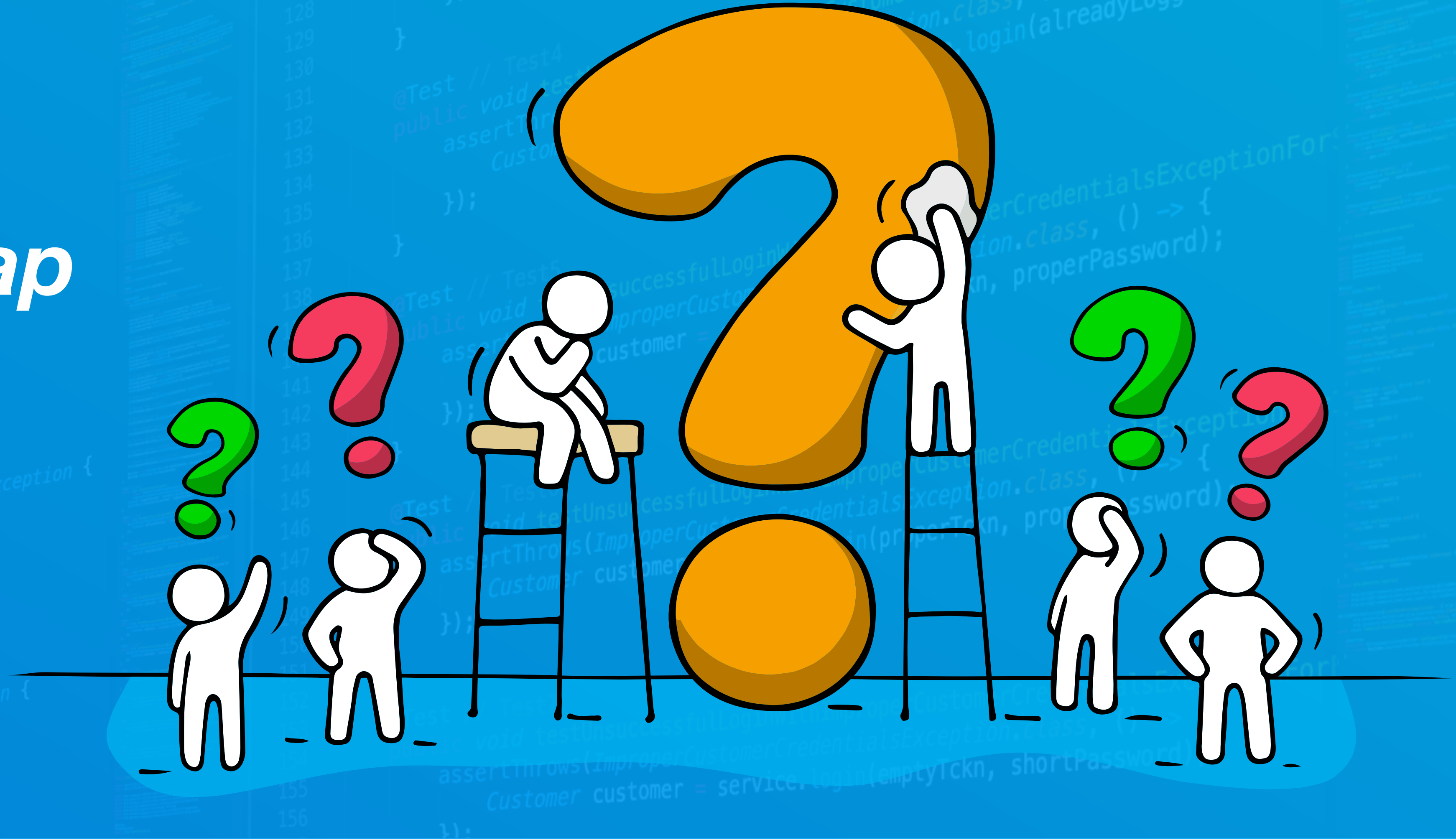
- `date.Date4` ve `DateTest`

Calendar



- `day.Calendar`

Soru ve Cevap Zamanı!





Diğer Özellikler

enum Sabitelerinin == ile Kıyaslanması



- **enum** sabiteleri birbirleriyle == ile kıyaslanabilir
- Kıyaslama için `equals ()` kullanmaya gerek yoktur çünkü her nesneden sadece bir tane vardır

```
enum Size {SMALL, MEDIUM, LARGE, EXTRA_LARGE}
```

```
Size.SMALL == Size.SMALL // true
```

switch-case'de enum Sabiteleri



- **enum** sabiteleri `switch-case` ifadesinde farklı değerlerine göre seçim yapmada kullanılabilir
- **enum** sabiteleri case ifadelerinde doğrudan, yani sınıf isimleri olmadan kullanılabilir

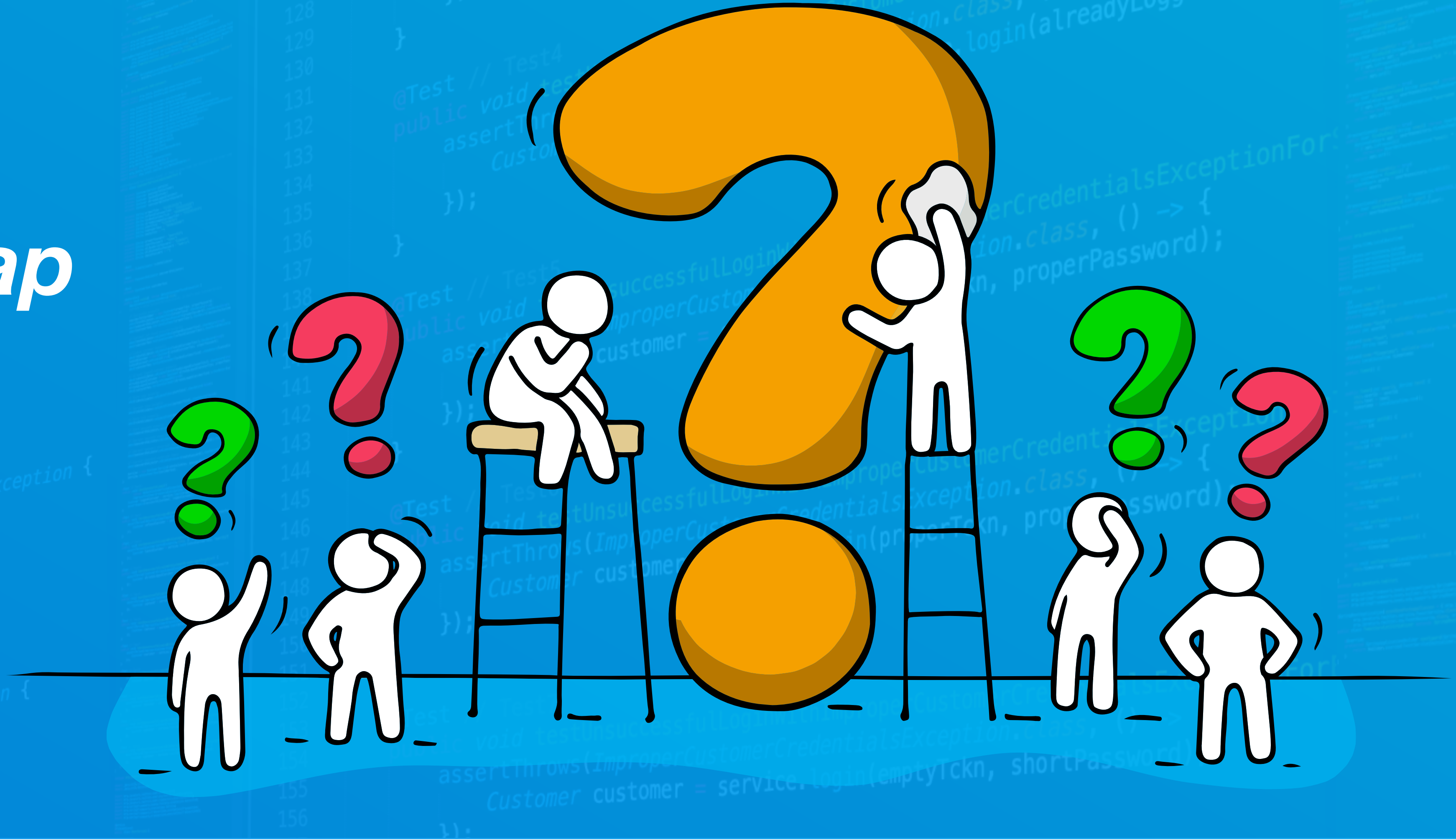
```
public void tellItLikeItIs() {  
  
    String message = switch (day) {  
        case MONDAY → "Mondays are bad.";  
        case FRIDAY → "Fridays are better.";  
        case SATURDAY, SUNDAY → "Weekends are best.";  
        default → "Midweek days are so-so.";  
    };  
  
    System.out.println(message);  
}
```

EnumSwitchDemo



- `day.EnumSwitchDemo`

Soru ve Cevap Zamanı!



Ödevler





1. Trafik ışıklarının renklerini, saniye olarak yanma süresine sahip `enum` ile betimleyin, test sınıfı yazın
 - a. Ayrıca yukarıda tanımladığınız trafik ışık rengi cinsinden bir duruma sahip, trafik ışığını temsil eden bir sınıf geliştirin
 - b. Bu sınıfa bir sonraki renge geçmeyi temsil eden bir metot koyun ve yeşil->sarı->kırmızı olacak şekilde çalışmasını sağlayın
 - a. Sizce bu metodun gerçekleştirilmesi enum sabiteler üzerinde başka bir metodu çağırarak şekilde tasarlanırsa nasıl olur?



2. `double basePrice` ve `int preparationTimeInMinutes` alanlarına sahip `small`, `medium`, `large` ve `family` pizza büyüklüklerini temsil eden `PizzaSize` isminde bir `enum` yapısı oluşturun
 - a. `PizzaSize` enum tipine `getPricePerSlice(int slices)` metodunu koyun ve bir pizzada 6 dilim olduğunu varsayarak, geçilen dilim sayısının fiyatını belirleyin
 - b. Benzer şekilde yapıya bir de `isGoodForParties()` metodunu ekleyin ve bu metot `large` ve `family` pizzaları için `true` döndürsün



- c. Pizzaların pişmesi için gereken süre ve sıcaklığı temsil etmek üzere iki alana sahip bir **PizzaCooking** sınıfı oluşturun
- d. Ayrıca, **PizzaOven** isimli bir sınıf daha oluşturun. Bu sınıfa koyacağınız bir metot **PizzaSize** argümanı alsın ve pizzanın pişmesi için gereken bilgiyi **PizzaCooking** olarak geri döndürsün. Bunun için **switch** expression kullanın
- e. **PizzaSizeTest** isimli **main** metoda sahip bir sınıfta bu yapıları test edin

Bundan Sonra?



Bundan Sonra?



- Bu, bu eğitimin Java'yı öğreten en son bölümüydü, bir sonraki bölüm, Java'yı öğrenmeye nasıl devam edebileceğimiz hakkında olacak:
- **Sırada Ne Var?**

Bölüm Sonu

Soru ve Cevap Zamanı!

